LAB 4

Simulated Internet & Private LANs with Stateless vs Stateful Services

0. Member

Student ID	Name	Role
673380416-3	นายพัทธดนย์ คำนัน	Network Architects
673380428-6	นายสิทธิโชค มุขนาค	IaC/DevOps
673380583-4	นายณัฐภัทร ฉ่ำตะคุ	QA/Test
673380606-8	นายสรวิศ สุคงเจริญ	QA/Test
673380608-4	นางสาวอมลวรรณ พิมพิชัย	Network Architects

1. Objective

วัตถุประสงค์ของ Lab นี้คือเพื่อศึกษาการทำงานของเครือข่ายคอมพิวเตอร์ในสภาพแวดล้อมที่มีหลาย LAN เชื่อมต่อกันผ่าน Router และ Simulated Internet รวมถึงศึกษาความแตกต่างระหว่าง **Stateless Services** และ **Stateful Services**

Lab นี้มีเป้าหมายหลักดังต่อไปนี้

1. สร้างโครงสร้างเครือข่ายที่ประกอบด้วย LAN สองฝั่ง
2. เชื่อมต่อเครือข่ายผ่าน Router ด้วย Static Routing
3. จำลอง Public Network (Internet) ด้วย subnet 10.10.12.0
4. ใช้ NAT (DNAT + MASQUERADE) เพื่อเข้าถึงบริการภายใน
5. ทดสอบ Stateless API และ Stateful API
6. วิเคราะห์การทำงานของ Session ผ่าน NAT

โดยโค้ดของ Lab 4 นักศึกษาได้จัดทำและอัพโหลดขึ้น GitHub เรียบร้อยแล้ว ท่านอาจารย์สามารถดูได้ผ่าน repo ของนักศึกษา ตามลิงค์ที่ได้แนบไว้นี้

GitHub link: https://github.com/ms584/CP352005-Computer-Networks-Laboratory/tree/main/network%20lab%204

2. Network Topology

โครงสร้างเครือข่ายที่ใช้ใน Lab มีดังนี้

```
              Simulated Internet
                 10.10.12.0/29
                       |
           --------------------------
           |                        |
        Router R1               Router R2
      10.10.12.1               10.10.12.2
           |                        |
      LAN A                      LAN B
   192.168.10.0/24          192.168.20.0/24
           |                        |
       -----------              -----------
       |         |              |         |
   ServerA   ClientA         ServerB   ClientB
192.168.10.10 192.168.10.50 192.168.20.10 192.168.20.50
```

เครือข่ายแบ่งออกเป็น 3 ส่วนคือ LAN A, LAN B, Simulated Internet และ Router R, R2 ที่ทำหน้าที่เป็น gateway สำหรับแต่ละ LAN

3. Address Table

Device	Interface	IP Address
R1	LAN	192.168.10.1
R1	Public	10.10.12.1
R2	LAN	192.168.20.1
R2	Public	10.10.12.2
ServerA	eth0	192.168.10.10
ClientA	eth0	192.168.10.50
ServerB	eth0	192.168.20.10
ClientB	eth0	192.168.20.50

4. Infrastructure Setup

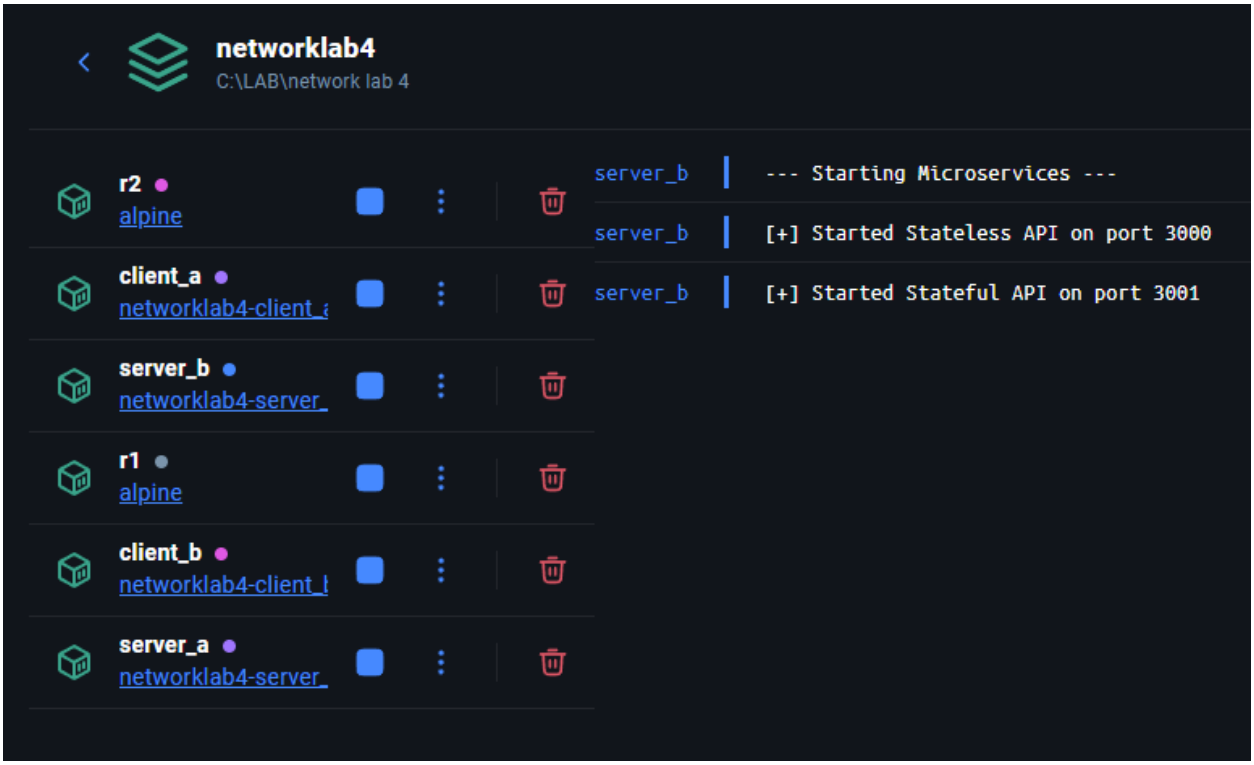
ระบบถูกสร้างโดยใช้ **Docker Compose** เพื่อจำลอง Router และ Host ภายในแต่ละ LAN

คำสั่งที่ใช้สร้างระบบ

docker compose down -v

docker compose up --build -d

หลังจากเริ่มระบบจะมี container ดังนี้



หลังจากเริ่มระบบด้วย docker compose up -d ได้ทำการตรวจสอบการทำงานของ Microservices ภายใน container ของ ServerA และ ServerB โดยใช้คำสั่ง docker logs ServerA และ docker logs ServerB

ผลลัพธ์ที่ได้แสดงดังภาพ ซึ่งมีข้อความดังรูป

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker logs ServerA
--- Starting Microservices ---
[+] Started Stateless API on port 3000
[+] Started Stateful API on port 3001

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker logs ServerB
--- Starting Microservices ---
[+] Started Stateless API on port 3000
[+] Started Stateful API on port 3001
```

ข้อความดังกล่าวยืนยันว่า **Microservices ภายใน container ถูกเริ่มต้นเรียบร้อยแล้ว** โดยมีการเปิดบริการ HTTP API สองประเภทคือ

- **Stateless API** ทำงานบน port **3000**
- **Stateful API** ทำงานบน port **3001**

Service ทั้งสองถูกเริ่มโดยสคริปต์ start_services.py ซึ่งทำหน้าที่สร้าง HTTP server สองตัวพร้อมกันผ่าน Python threading เพื่อจำลอง microservices ภายในระบบ

5. Routing Configuration

Router ทั้งสองตัวใช้ **Static Routing**

R1: 192.168.20.0/24 via 10.10.12.2

R2: 192.168.10.0/24 via 10.10.12.1

ตรวจสอบด้วยคำสั่ง docker exec R1 ip route และ docker exec R2 ip route

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec R1 ip route
default via 192.168.10.254 dev eth0
10.10.12.0/29 dev eth1 proto kernel scope link src 10.10.12.1
192.168.10.0/24 dev eth0 proto kernel scope link src 192.168.10.1
192.168.20.0/24 via 10.10.12.2 dev eth1

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec R2 ip route
default via 192.168.20.254 dev eth0
10.10.12.0/29 dev eth1 proto kernel scope link src 10.10.12.2
192.168.10.0/24 via 10.10.12.1 dev eth1
192.168.20.0/24 dev eth0 proto kernel scope link src 192.168.20.1
```

ผลลัพธ์แสดงว่า router สามารถ route traffic ข้าม LAN ได้

6. NAT Configuration

Router R2 ถูกตั้งค่าให้ทำ **DNAT** เพื่อให้ Public Network สามารถเข้าถึง ServerB ได้

DNAT Rule

10.10.12.2:3000 -> 192.168.20.10:3000

10.10.12.2:3001 -> 192.168.20.10:3001

iptables configuration

PREROUTING

DNAT tcp dpt:3000 -> 192.168.20.10:3000

DNAT tcp dpt:3001 -> 192.168.20.10:3001

POSTROUTING

MASQUERADE

ตรวจสอบด้วยคำสั่ง

docker exec R2 iptables -t nat -L -n -v

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec R2 iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT       tcp  --  eth1   *       0.0.0.0/0            0.0.0.0/0            tcp dpt:3000 to:192.168.20.10:3000
    0     0 DNAT       tcp  --  eth1   *       0.0.0.0/0            0.0.0.0/0            tcp dpt:3001 to:192.168.20.10:3001

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  tcp  --  *      eth0    0.0.0.0/0            192.168.20.10        tcp dpt:3000
    0     0 MASQUERADE  tcp  --  *      eth0    0.0.0.0/0            192.168.20.10        tcp dpt:3001

Chain DOCKER_OUTPUT (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER_POSTROUTING (0 references)
 pkts bytes target     prot opt in     out     source               destination
```

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec R2 iptables -L FORWARD -n -v
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     tcp  --  eth1   eth0    0.0.0.0/0            192.168.20.10        tcp dpt:3000 ctstate NEW,RELATED,ESTABLISHED
    0     0 ACCEPT     tcp  --  eth1   eth0    0.0.0.0/0            192.168.20.10        tcp dpt:3001 ctstate NEW,RELATED,ESTABLISHED
    0     0 ACCEPT     all  --  eth0   eth1    0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
```

7. Connectivity Test

ทดสอบการเชื่อมต่อระหว่าง LAN

ClientA → ServerB: docker exec ClientA ping 192.168.20.10

ผล 0% packet loss

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA ping -c 3 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=62 time=0.138 ms
64 bytes from 192.168.20.10: seq=1 ttl=62 time=0.113 ms
64 bytes from 192.168.20.10: seq=2 ttl=62 time=0.086 ms

--- 192.168.20.10 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.086/0.112/0.138 ms
```

ServerB → ClientA: docker exec ServerB ping 192.168.10.50

ผล 0% packet loss

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ServerB ping -c 3 192.168.10.50
PING 192.168.10.50 (192.168.10.50): 56 data bytes
64 bytes from 192.168.10.50: seq=0 ttl=62 time=0.100 ms
64 bytes from 192.168.10.50: seq=1 ttl=62 time=0.109 ms
64 bytes from 192.168.10.50: seq=2 ttl=62 time=0.103 ms

--- 192.168.10.50 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.100/0.104/0.109 ms
```

แสดงว่า routing ระหว่าง LAN ทำงานถูกต้อง

8. Local Service Testing (Private LAN)

ก่อนทำการทดสอบผ่าน Simulated Internet ได้มีการทดสอบบริการภายในเครือข่าย LAN A ก่อน เพื่อยืนยันว่า Application และ Microservices ทำงานได้ถูกต้องก่อนที่จะส่ง traffic ผ่าน Router และ NAT การทดสอบนี้ทำจาก ClientA (192.168.10.50) ไปยัง ServerA (192.168.10.10) ภายในเครือข่ายเดียวกัน

คำสั่งที่ใช้ทดสอบ

docker exec -it ClientA sh -c "curl -v http://192.168.10.10:3000/info; echo"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA sh -c "curl -v http://192.168.10.10:3000/info; echo"
*   Trying 192.168.10.10:3000...
* Established connection to 192.168.10.10 (192.168.10.10 port 3000) from 192.168.10.50 port 34914
* using HTTP/1.x
> GET /info HTTP/1.1
> Host: 192.168.10.10:3000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 03:00:26 GMT
< Content-Type: application/json
< Content-Length: 97
<
* shutting down connection #0
{"status": "healthy", "service": "Stateless API", "port": 3000, "server_time": 1772766026.983487}
```

การวิเคราะห์ผล

คำสั่ง curl ส่ง HTTP GET request ไปยัง endpoint /info

Server ตอบกลับด้วยสถานะ: HTTP/1.0 200 OK

พร้อม JSON response ที่แสดงว่า

- Service ทำงานปกติ
- Stateless API ไม่ต้องใช้ session
- ทุก request ทำงานอิสระ

Packet flow ใน Local Test

```
ClientA (192.168.10.50)
        |
        ▼
Switch / LAN A
        |
        ▼
ServerA (192.168.10.10)
```

เนื่องจากอยู่ใน LAN เดียวกัน packet จึง **ไม่ผ่าน router**

8.2 Local Stateful API Test

Stateful API ทำงานบน **port 3001**

การทดสอบ Stateful ต้องมี 2 ขั้นตอน

1. สร้าง session
2. ใช้ session

Step 1: Create Session

คำสั่ง docker exec -it ClientA sh -c "curl -v -X POST http://192.168.10.10:3001/session; echo"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA sh -c "curl -v -X POST http://192.168.10.10:3001/session; echo"
*   Trying 192.168.10.10:3001...
* Established connection to 192.168.10.10 (192.168.10.10 port 3001) from 192.168.10.50 port 44360
* using HTTP/1.x
> POST /session HTTP/1.1
> Host: 192.168.10.10:3001
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 03:00:48 GMT
< Content-Type: application/json
< Content-Length: 132
<
* shutting down connection #0
{"status": "success", "message": "Session created", "service": "Stateful API", "session_id": "dea13efc-823d-4235-986f-0ed58b3c2779"}
```

การวิเคราะห์

Server สร้าง session ใหม่และส่งค่า **session_id** กลับมา Session นี้ถูกเก็บไว้ใน **memory ของ server**

Step 2: Access Session Data

นำ session_id ที่ได้จากขั้นตอนแรกไปใช้ใน request ถัดไป

คำสั่ง docker exec -it ClientA sh -c "curl -v http://192.168.10.10:3001/data -H 'Session-ID: "dea13efc-823d-4235-986f-0ed58b3c2779"'; echo"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA sh -c "curl -v http://192.168.10.10:3001/data -H 'Session-ID: "dea13efc-823d-4235-986f-0ed58b3c2779"'; echo"
*   Trying 192.168.10.10:3001...
* Established connection to 192.168.10.10 (192.168.10.10 port 3001) from 192.168.10.50 port 37668
* using HTTP/1.x
> GET /data HTTP/1.1
> Host: 192.168.10.10:3001
> User-Agent: curl/8.17.0
> Accept: */*
> Session-ID: dea13efc-823d-4235-986f-0ed58b3c2779
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 03:02:22 GMT
< Content-Type: application/json
< Content-Length: 211
<
* shutting down connection #0
{"status": "success", "service": "Stateful API", "session_id": "dea13efc-823d-4235-986f-0ed58b3c2779", "session_data": {"created_at": 1772766048.1718209, "client_ip": "192.168.10.50
", "service": "Stateful API"}}
```

การวิเคราะห์ผล

Server สามารถ

- ตรวจสอบ Session-ID
- ดึงข้อมูล session จาก memory
- ส่ง session data กลับมาได้

แสดงว่า **state ถูกเก็บและใช้งานได้จริง**

8.3 Stateless vs Stateful Behavior (Local)

Feature	**Stateless API**	**Stateful API**
Session required	No	Yes
Request independence	Yes	No
Server memory usage	No	Yes
Headers required	None	Session-ID

8.4 Importance of Local Testing

การทดสอบภายใน LAN ก่อนมีความสำคัญ เพราะช่วยยืนยันว่า

1. Application ทำงานถูกต้อง
2. Microservices เริ่มทำงานแล้ว
3. Ports เปิดถูกต้อง
4. API endpoints ใช้งานได้

หลังจากยืนยันว่า service ทำงานใน LAN ได้แล้ว จึงทำการทดสอบ **Remote Access ผ่าน Router และ NAT**

8.5 Summary of Local Tests

ผลการทดสอบภายใน LAN แสดงว่า

- Stateless API สามารถตอบ request ได้ทันที
- Stateful API สามารถสร้างและใช้ session ได้
- Application ทำงานถูกต้องก่อนส่ง traffic ผ่าน network layer

ดังนั้นระบบพร้อมสำหรับการทดสอบ **Simulated Internet Access ผ่าน Router และ NAT**

9. Stateless Service Test

Stateless API ใช้ port **3000**

ทดสอบจาก ClientA ผ่าน simulated internet

docker exec -it ClientA sh -c "curl -v --connect-timeout 3 --max-time 5 -X POST http://10.10.12.2:3001/session; echo"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA sh -c "curl -v --connect-timeout 3 --max-time 5 http://10.10.12.2:3000/info; echo"
*   Trying 10.10.12.2:3000...
* Established connection to 10.10.12.2 (10.10.12.2 port 3000) from 192.168.10.50 port 32874
* using HTTP/1.x
> GET /info HTTP/1.1
> Host: 10.10.12.2:3000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 03:02:41 GMT
< Content-Type: application/json
< Content-Length: 98
<
* shutting down connection #0
{"status": "healthy", "service": "Stateless API", "port": 3000, "server_time": 1772766161.3239403}
```

คุณสมบัติของ Stateless Service

- ไม่เก็บ session
- แต่ละ request ทำงานอิสระ
- ไม่ต้องใช้ header เพิ่มเติม

10. Stateful Service Test

Stateful API ใช้ port **3001**

Step 1: Create Session

docker exec -it ClientA sh -c "curl -v --connect-timeout 3 --max-time 5 -X POST http://10.10.12.2:3001/session; echo"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA sh -c "curl -v --connect-timeout 3 --max-time 5 -X POST http://10.10.12.2:3001/session; echo"
*   Trying 10.10.12.2:3001...
* Established connection to 10.10.12.2 (10.10.12.2 port 3001) from 192.168.10.50 port 34438
* using HTTP/1.x
> POST /session HTTP/1.1
> Host: 10.10.12.2:3001
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 03:02:47 GMT
< Content-Type: application/json
< Content-Length: 132
<
* shutting down connection #0
{"status": "success", "message": "Session created", "service": "Stateful API", "session_id": "5dad983c-af7c-4974-9cc2-54dc43f56483"}
```

Step 2: Access Session Data

docker exec -it ClientA sh -c "curl -v --connect-timeout 3 --max-time 5 http://10.10.12.2:3001/data -H 'Session-ID: "5dad983c-af7c-4974-9cc2-54dc43f56483"'; echo"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 4
$ docker exec -it ClientA sh -c "curl -v --connect-timeout 3 --max-time 5 http://10.10.12.2:3001/data -H 'Session-ID: "5dad983c-af7c-4974-9cc2-54dc43f56483"'; echo"
*   Trying 10.10.12.2:3001...
* Established connection to 10.10.12.2 (10.10.12.2 port 3001) from 192.168.10.50 port 44012
* using HTTP/1.x
> GET /data HTTP/1.1
> Host: 10.10.12.2:3001
> User-Agent: curl/8.17.0
> Accept: */*
> Session-ID: 5dad983c-af7c-4974-9cc2-54dc43f56483
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 03:03:10 GMT
< Content-Type: application/json
< Content-Length: 208
<
* shutting down connection #0
{"status": "success", "service": "Stateful API", "session_id": "5dad983c-af7c-4974-9cc2-54dc43f56483", "session_data": {"created_at": 1772766167.65986, "client_ip": "192.168.20.1",
"service": "Stateful API"}}
```

11. Packet Flow Analysis

เมื่อ ClientA เรียก curl http://10.10.12.2:3000/info packet flow จะเป็นดังนี้

```
ClientA
192.168.10.50
        |
        v
Router R1
192.168.10.1
        |
        v
Simulated Internet
10.10.12.0/29
        |
        v
Router R2
10.10.12.2
        |
        v
DNAT
192.168.20.10
        |
        v
ServerB
```

Response จะถูกส่งกลับผ่าน path เดิมโดยใช้ MASQUERADE

12. Stateless vs Stateful Comparison

Feature	**Stateless**	**Stateful**
Session storage	No	Yes
Request independence	Yes	No
Headers required	No	Session-ID
Server memory	Not required	Required
Scalability	High	Lower

13. Engineering Reflection

จากการทดลองพบว่า

1. Stateless services ไม่ต้องเก็บ session ทำให้ระบบสามารถ scale ได้ง่าย
2. Stateful services ต้องใช้ session management
3. NAT ไม่ส่งผลต่อ logical session identity
4. DNAT สามารถใช้เพื่อ expose service ภายใน network

14. Conclusion

Lab นี้แสดงให้เห็นการทำงานของระบบเครือข่ายที่ประกอบด้วย

- Static routing
- Network Address Translation (NAT)
- Multi-LAN topology
- Microservice architecture

ผลการทดลองแสดงว่า

- Router สามารถ route traffic ระหว่าง LAN ได้
- NAT สามารถ expose internal service ผ่าน public interface ได้
- Stateless และ Stateful services มีพฤติกรรมแตกต่างกันอย่างชัดเจน